Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	07 March 2026
Team ID	Team 295
Project Name	UCab – Cab Booking Platform
Maximum Marks	4 Marks

Technical Architecture:

The **UCab Cab Booking Platform** is implemented as a web-based system that connects **riders, drivers, and administrators** through a centralized application. The platform allows riders to request rides, drivers to manage trip requests, and administrators to monitor and manage the overall system.

The system follows a **client–server architecture**, where the **frontend interface** allows users to interact with the platform through dashboards and booking features, while the **backend server** handles business logic such as authentication, ride requests, driver matching, payment processing, and trip management.

All application data, including **user accounts, driver information, ride records, and payment details**, is stored in a **database**. APIs are used to enable communication between the frontend and backend services. The system also manages **real-time ride updates, trip status changes, and feedback storage** to ensure smooth operation of the cab booking service.

This technical structure ensures that the UCab platform is **secure, scalable, and efficient**, allowing multiple users to interact with the system simultaneously while maintaining reliable ride booking and management.

Technical Architecture – UCab Cab Booking Platform

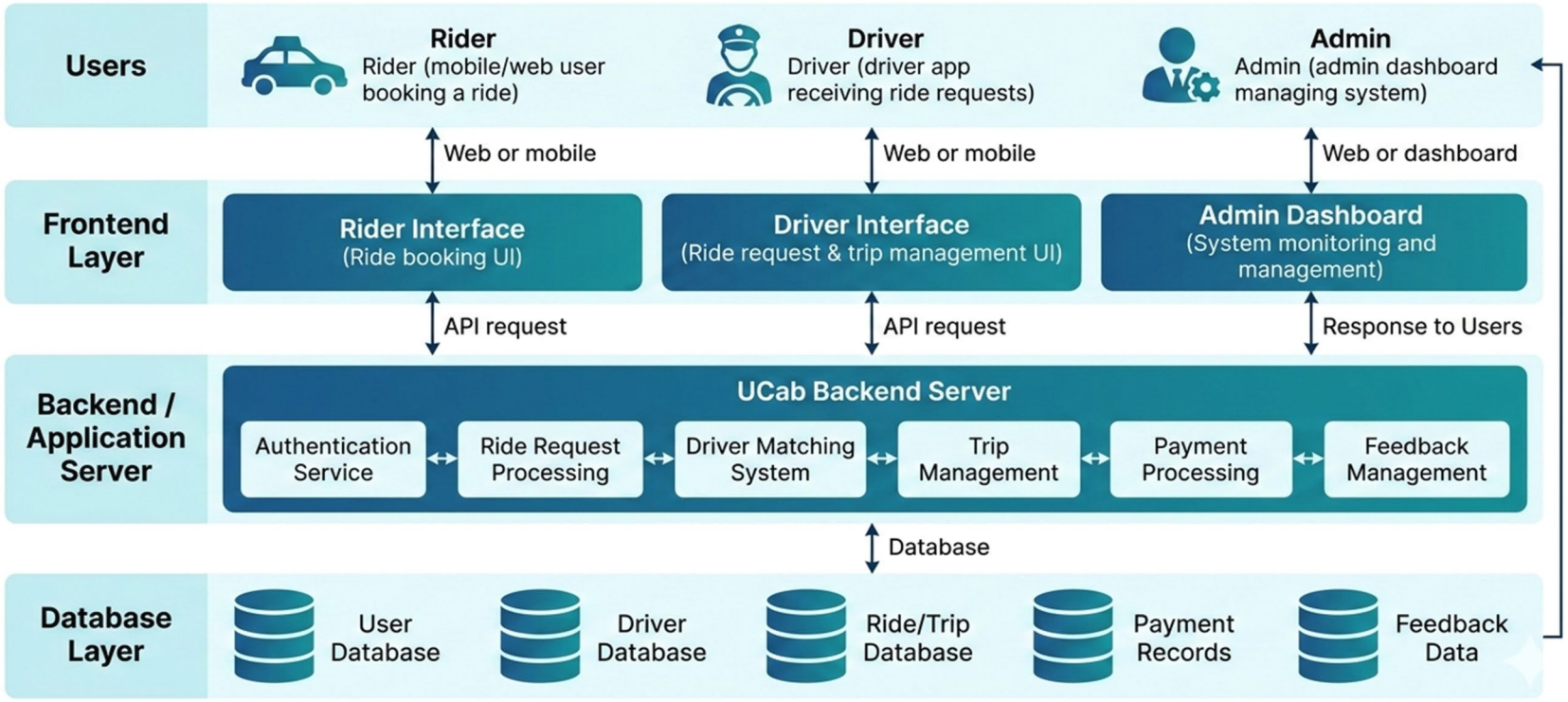

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Interface through which riders, drivers, and admins interact with the system (web dashboard / booking screens).	HTML, CSS, JavaScript / Angular / React etc.
2.	Application Logic	Handles business logic such as authentication, ride booking, and user management.	Node.js / Express
3.	Ride Management Service	Processes ride requests, driver matching, trip status updates, and ride completion.	Backend APIs
4.	Authentication & Security	Manages login, session handling, and secure access control.	JWT Authentication
5.	Database	Stores user profiles, driver details, ride records, and feedback data.	MongoDB / MySQL
6.	File Storage	Stores profile images, documents, or other uploaded files.	Local Storage / Cloud Storage
7.	External API	Used for map services, location tracking, and route information.	Google Maps API
8.	Payment Integration	Handles ride payment processing and transaction records.	Payment Gateway API
9.	Infrastructure (Server / Cloud)	Deployment environment for hosting the application and backend services.	Local Server / Cloud Hosting (Docker / Cloud Platform)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	The application is developed using open-source frameworks to build the frontend interface and backend services efficiently.	React, Node.js, Express
2.	Security Implementations	Secure authentication, protected API routes, and encrypted password storage to protect user data and system access.	JWT Authentication, Password Hashing, HTTPS
3.	Scalable Architecture	The system follows a layered architecture separating frontend, backend, and database components to support future scaling.	Client–Server / 3-Tier Architecture
4.	Availability	The platform is designed to remain accessible to users with reliable server deployment and database connectivity.	Cloud Hosting / Web Server Deployment
5.	Performance	Efficient API communication and optimized database queries ensure fast ride booking and data processing.	REST APIs, Optimized Database Queries